

What We'll Cover

1

Prim's Algorithm

Building Minimum Spanning Trees from scratch

2

Round Robin

Fair and efficient CPU scheduling

3

Complexity Analysis

Real-world performance and trade-offs

4

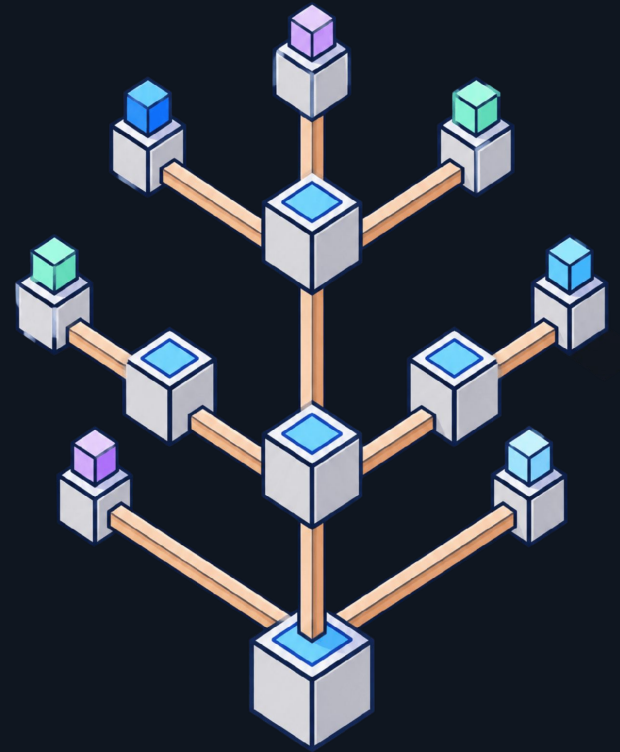
Visual Walkthroughs

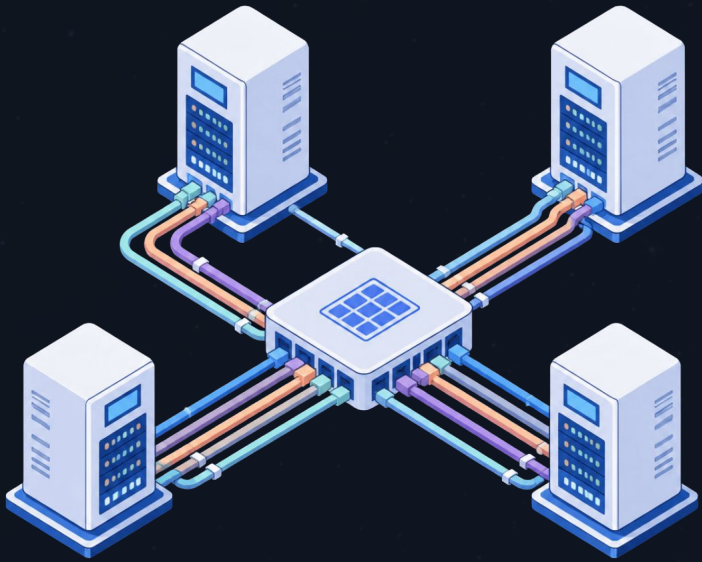
Step-by-step examples and comparisons

PART 1

Prim's Algorithm

Greedy graph traversal for minimum spanning
trees





What is a Minimum Spanning Tree?

A **Minimum Spanning Tree (MST)** is a subgraph that connects all vertices in a weighted graph using the minimum possible total edge weight and with no cycles.

All Vertices Connected

Every node in the graph is reachable from any other node.

Minimum Total Weight

The sum of all edge weights is as small as possible.

No Cycles

It's a tree, adding any edge would create a cycle.

Prim's Algorithm Overview



Pick a Start Vertex

Any vertex can be chosen as the starting point of the MST.



Examine Edges

Scan all edges connecting visited to unvisited vertices.



Pick the Minimum

Greedy select the lowest-weight edge each iteration.



Grow the Tree

Repeat until all vertices are included in the MST.

❏ **Key Principle:** Always pick the smallest edge that connects the growing tree to a new, unvisited vertex and never revisit a node.

How Prim's Works: Steps 1 & 2



Choose Start

Record
Edges

Add Min
Edge

Each iteration expands the MST by exactly one vertex and one edge, guaranteeing no cycles are formed.

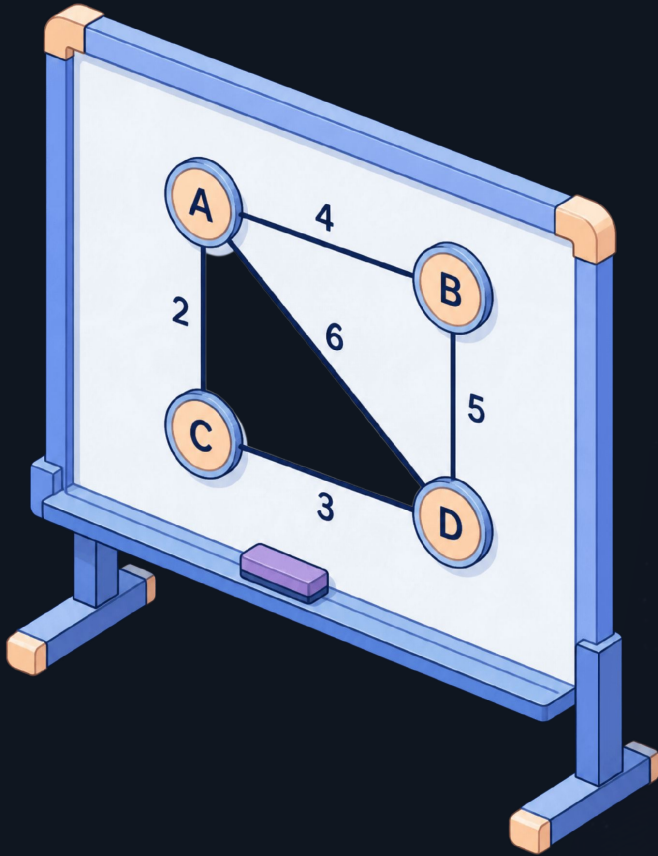
Step 1 — Initialize

- **Choose a starting vertex** (any vertex works)
- **Mark it as visited** and add it to the MST
- **Examine all outgoing edges** from this vertex

Step 2 — Expand

- **Find the minimum-weight edge** to an unvisited vertex
- **Add that edge and its destination vertex** to the MST
- **Repeat until all vertices are included**

Prim's Algorithm: Worked Example



01) Start at A

Add edges **AB(1)**, AC(4), AD(2) to consideration.

02) Pick AB(1)

Cheapest edge. Add vertex B. New candidates: BC(3), BD(5).

03) Pick AD(2)

Next cheapest. Add vertex D. New candidate: DC(1).

04) Pick DC(1)

Add vertex C. All 4 vertices included. **MST complete!** Total weight = **4**.

Prim's Complexity Analysis

Array Implementation

Time: $O(V^2)$

Simple to implement. Efficient for dense graphs where $E \approx V^2$.

Min-Heap Implementation

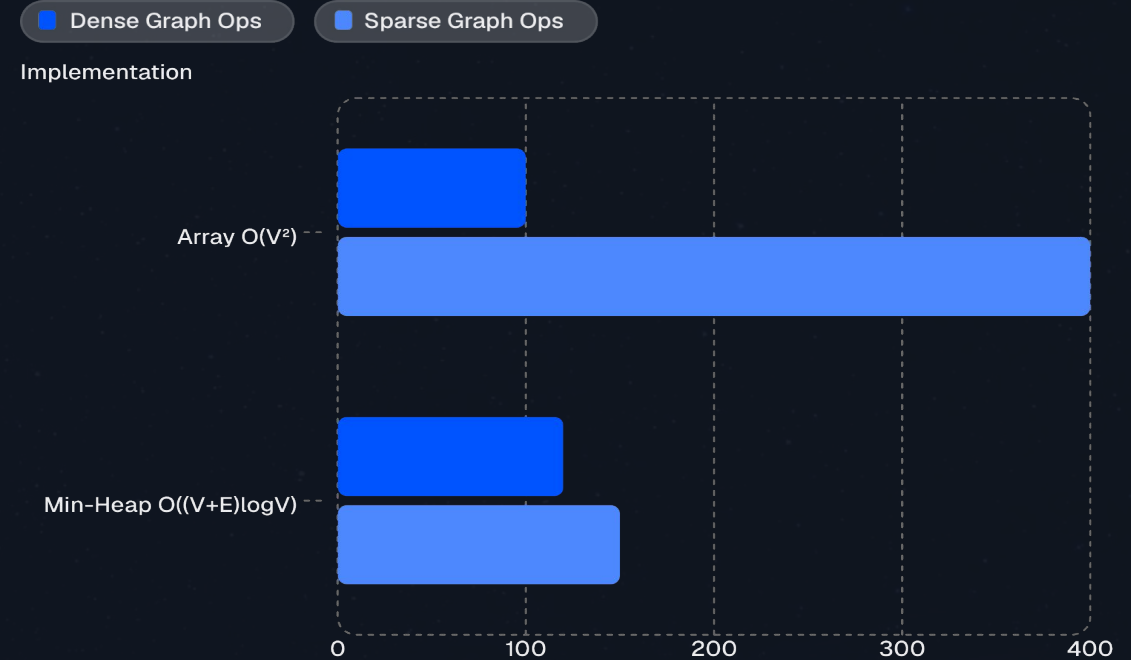
Time: $O((V + E) \log V)$

Faster for sparse graphs. Uses a priority queue for edge selection.

Space Complexity

Space: $O(V)$

Tracks visited vertices and the current minimum key values.



Relative operation count (lower = faster). Min-heap wins on sparse graphs; array is competitive on dense ones.

Prim's vs. Kruskal's Algorithm



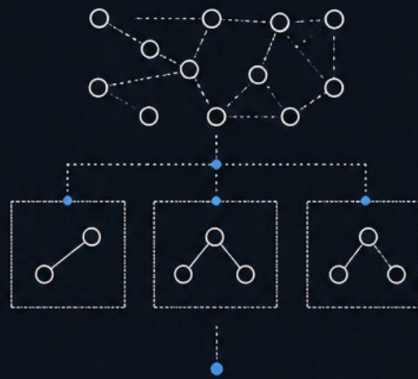
Prim's Algorithm

starts from a single vertex

grows the tree edge by edge

ideal for dense graphs

$O(V^2)$ with array



Kruskal's Algorithm

sorts all edges globally by weight

uses union-find to avoid cycles

ideal for sparse graphs

$O(E \log E)$

What They Share

- Both are **greedy algorithms** guaranteed to produce a valid MST
- Both yield the **same minimum total weight**, though edge order may differ
- Both run efficiently on typical graph sizes encountered in practice

When to Choose Prim's

- **Dense graphs** where edges $\gg$ vertices
- When adjacency matrix representation is used
- When growing a connected structure incrementally matters

Real-World Applications of Prim's Algorithm



Network Design

Minimize total cable length when connecting computers, switches, and routers in a LAN or data center.



Road & Rail Networks

Connect cities or stations with the least total road or track distance, reducing construction costs.



Data Clustering

Group similar data points by removing the heaviest MST edges to form natural clusters in ML pipelines.



Key Takeaways

MST Fundamentals

A Minimum Spanning Tree connects all vertices with the smallest total edge weight and zero cycles.

Prim's Strategy

Greedy, vertex-centric growth. Always extend the tree with the cheapest available edge to an unvisited node.

Complexity

$O(V^2)$ with arrays for dense graphs; $O((V+E) \log V)$ with a min-heap for sparse graphs.

Applications

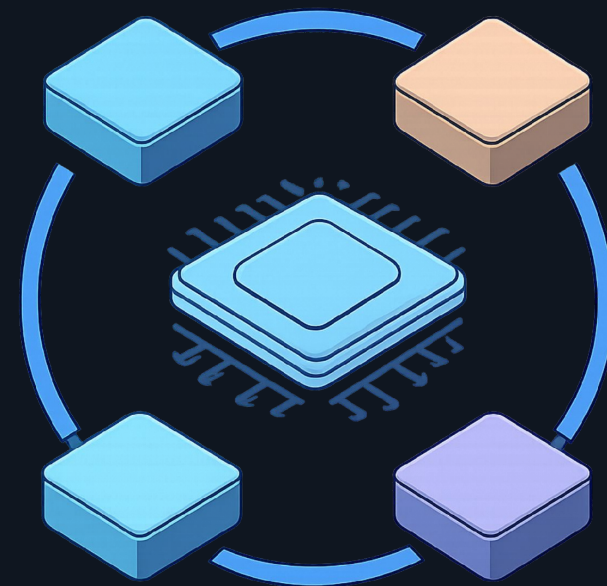
Network cabling, road planning, and data clustering all rely on MST-based approaches in practice.

📌 Mastering Prim's Algorithm builds the intuition for greedy design patterns — a foundational skill for advanced DSA topics.

PART 2

Round Robin Scheduling

Fair and efficient CPU process management



What is CPU Scheduling?

CPU scheduling is the process of determining which of the many ready processes in the operating system should be executed by the CPU at any given time. It's a fundamental OS function that aims to balance resource utilization with user experience.

Maximize CPU Utilization

Keep the processor busy as much as possible to avoid idle time.

Ensure Fairness

Provide equitable access to CPU resources for all competing processes.

Minimize Response Time

Reduce the delay between a user request and the system's first reaction.

Maximize Throughput

Complete as many processes as possible within a given time period.

Round Robin Scheduling: The Overview

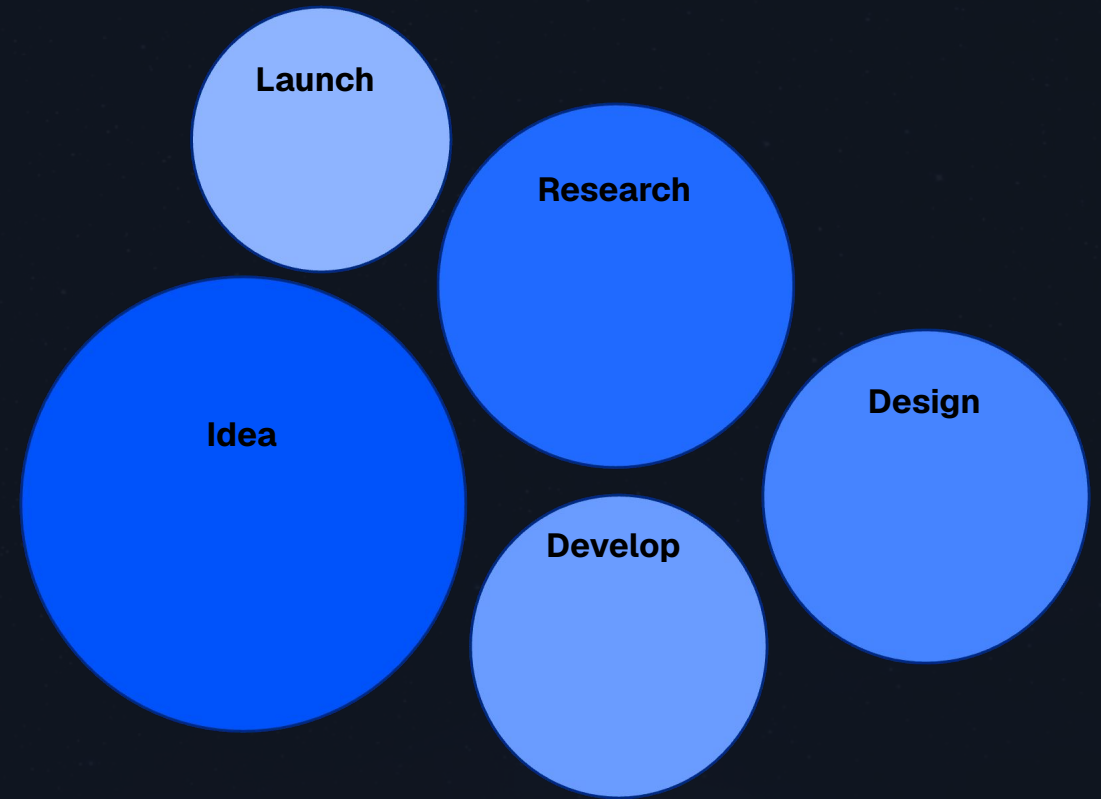
Round Robin is a preemptive CPU scheduling algorithm designed for time-sharing systems. It's one of the simplest and most widely used scheduling algorithms, ensuring fairness by giving each process a small, fixed unit of CPU time, called a **time quantum**. This cyclical approach, often implemented with a **circular queue**, prevents any single process from monopolizing the CPU, ensuring all processes get a fair share of processing time.

Preemptive Scheduling

Processes can be interrupted and removed from the CPU before completion, allowing for dynamic allocation and responsiveness.

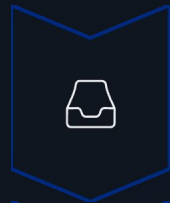
Time-Sharing Systems

Designed to give the illusion of multiple processes running simultaneously, enhancing user experience in interactive and multi-user environments.



How Round Robin Works

Round Robin (RR) is a CPU scheduling algorithm that fairly shares CPU time among multiple processes. It's designed for time-sharing systems where each process gets a small unit of CPU time, called a **time quantum**.



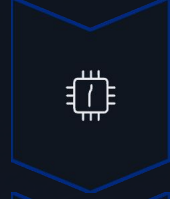
Process Enters Queue

New processes are added to the end of the ready queue, waiting for CPU allocation.



Allocate Time Quantum

The CPU picks the first process from the queue and assigns it a fixed time quantum.



Execute Process

The process runs for the duration of its time quantum, or until it completes, whichever comes first.



Preempt & Requeue

If the process does not finish, it's preempted and moved to the back of the ready queue.



Cycle Continues

The scheduler moves to the next process, repeating the cycle until all tasks are completed.

Round Robin Scheduling: Worked Example

Let's walk through a classic Round Robin (RR) scheduling scenario with a time quantum of **2ms**.

Processes & Burst Times

- P1: 6ms
- P2: 8ms
- P3: 7ms
- P4: 3ms

Next (2-4ms): P2

P2 executes for 2ms. P2 remaining: 6ms. Ready Queue: [P3, P4, P1, P2].

Next (6-8ms): P4

P4 executes for 2ms. P4 remaining: 1ms. Ready Queue: [P1, P2, P3, P4].

Start (0-2ms): P1

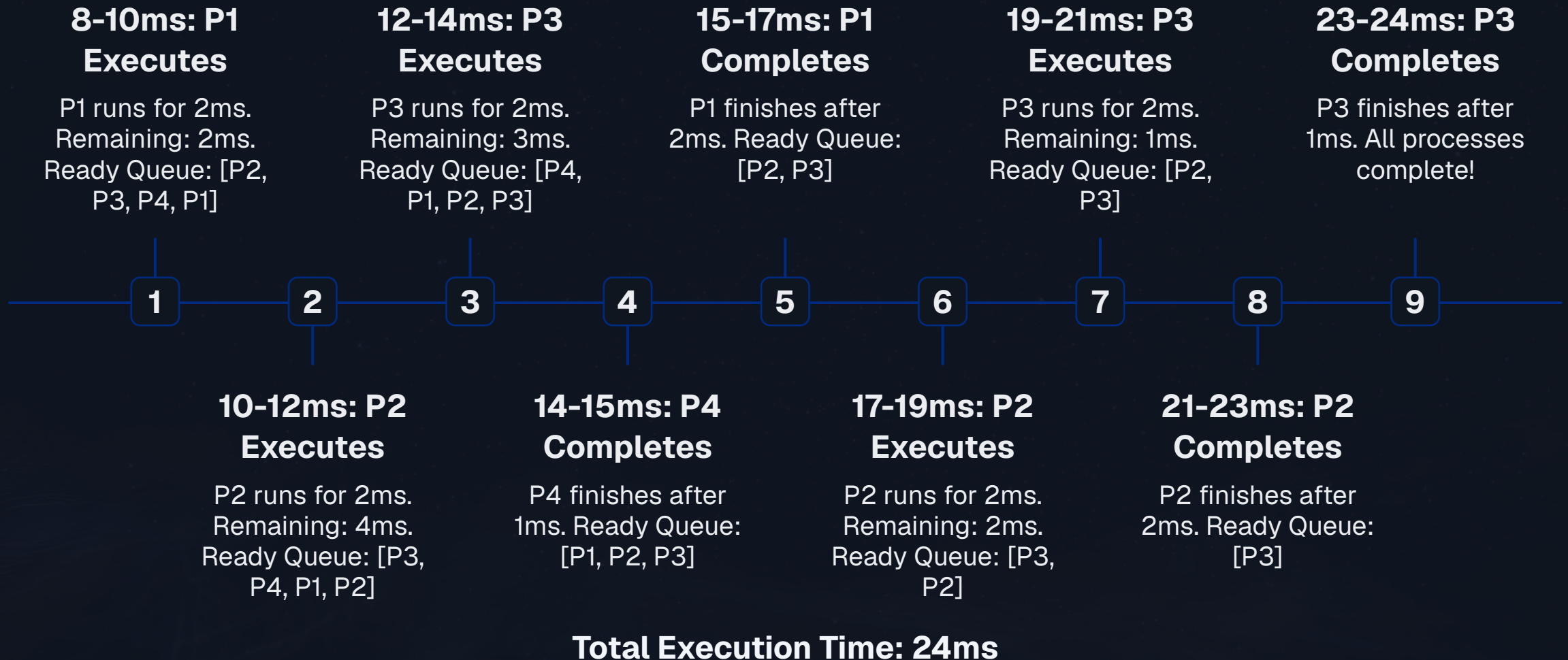
P1 executes for 2ms. P1 remaining: 4ms. Ready Queue: [P2, P3, P4, P1].

Next (4-6ms): P3

P3 executes for 2ms. P3 remaining: 5ms. Ready Queue: [P4, P1, P2, P3].

Round Robin Example: Execution Timeline (Part 2)

Continue the Round Robin scheduling example with time quantum = 2ms.



Round Robin Complexity Analysis



Time Complexity

Primarily impacted by the number of processes (N) and the time quantum (Q).

A smaller Q results in more context switches, increasing overhead.



Space Complexity

Typically $O(N)$ for storing process control blocks (PCBs) in the ready queue.

Each process requires space to save its current state.



Context Switch Overhead

Each switch between processes incurs a cost (saving CPU state, loading new state).

The optimal time quantum balances system responsiveness with minimizing this overhead.

Understanding these factors is crucial for configuring Round Robin scheduling efficiently in real-world operating systems.

Round Robin vs. Other Schedulers

Understanding the trade-offs between different CPU scheduling algorithms is crucial for designing efficient operating systems. Let's compare three fundamental approaches:

Criterion	FCFS (First-Come, First-Served)	SJF (Shortest Job First)	Round Robin
Approach	Queue-based on arrival time.	Prioritizes shortest burst time.	Time-sliced, pre-emptive.
Preemptive	No	No (in basic form)	Yes
Fairness	High (by arrival)	Low (can starve long jobs)	High (equal CPU share)
Starvation	No	Yes (for long jobs)	No
Context Switch Overhead	Low	Low	High (per time quantum)
Best Use Case	Batch systems	Jobs with known, short burst times	Interactive systems, time-sharing

Each algorithm has strengths and weaknesses, making the choice dependent on system requirements for throughput, turnaround time, waiting time, and response time.



Real-World Applications of Round Robin

Operating Systems

Ensures fair allocation of CPU time among multiple processes, preventing any single process from monopolizing resources and avoiding starvation.

Web Servers

Distributes incoming client requests evenly across a pool of servers, optimizing load balancing and enhancing overall system responsiveness and throughput.

Gaming

Manages turns for multiple players in online games or schedules various game logic components (AI, physics) in a cyclical fashion for smooth gameplay.

Key Takeaways: Prim's Algorithm & Round Robin



Prim's: MST Creation

Greedy builds a Minimum Spanning Tree by adding the cheapest edge to an unvisited vertex, ensuring minimal total weight and no cycles.



Prim's: Complexity

Performance varies: $O(V^2)$ for array implementations (dense graphs) and $O((V+E)\log V)$ with a min-heap (sparse graphs).



Prim's: Applications

Used in network design, road construction, and data clustering to optimize connections and minimize costs.



Round Robin: Time Slicing

Ensures fair CPU allocation by giving each process a fixed time quantum, preventing starvation and promoting responsiveness in multi-tasking systems.



Round Robin: Overhead

Introduces context switching overhead. Optimal quantum size is crucial: too small increases overhead, too large reduces fairness.



Round Robin: Use Cases

Ideal for time-sharing operating systems where quick response times for multiple users are paramount.

Thank

You!

We've explored Prim's Algorithm for MSTs and briefly touched on related concepts.

Questions?

Your feedback and questions are highly valuable for refining our understanding.

Connect with Us

Explore more topics at our website or reach out directly.

Provide Feedback

Scan the QR code to share your thoughts on this session.

